

Student 23 **Perarasan V** 192521404RD

10 / 10 submitted

Q1. Selection Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def selection_sort(arr):
    for i in range(len(arr)):
        m = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[m]:
                m = j
        arr[i], arr[m] = arr[m], arr[i]
    return arr

data = [int(x) for x in input().split()]
print(selection_sort(data))
```

INPUT

4 2 8 6 1

OUTPUT

[1, 2, 4, 6, 8]

Q2. Insertion Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j ≥ 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
```

```
arr[j + 1] = key  
return arr
```

```
print(insertion_sort(list(map(int, input().split()))))
```

INPUT

4 1 3 2

OUTPUT

[1, 2, 3, 4]

Q3. Merge Sort

Score: 10.00 TC: 3/3 passed

CODE

```
def merge_sort(arr):
    if len(arr) ≤ 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

if __name__ == "__main__":
    try:
        data = list(map(int, input().split()))
        print(merge_sort(data))
    except EOFError:
        pass
```

INPUT

38 27 43 3 9 82 10

OUTPUT

[3, 9, 10, 27, 38, 43, 82]

Q4. Diagonal Matrix

CODE

```
def is_diagonal_matrix(mat):  
    n = len(mat)  
    for i in range(n):  
        for j in range(len(mat[i])):  
            if i != j and mat[i][j] != 0:  
                return "Not Diagonal Matrix"  
    return "Diagonal Matrix"  
  
# Input format: matrix as a Python list of lists  
mat = eval(input().strip())  
print(is_diagonal_matrix(mat))
```

INPUT

```
[[1,0,0],[0,5,0],[0,0,8]]
```

OUTPUT

```
(no output)
```

Q5. Row and Column Sums

CODE

```
import ast
matrix = ast.literal_eval(input())
row_sums = [sum(row) for row in matrix]
col_sums = [sum(row[i] for row in matrix) for i in range(len(matrix[0]))]
print(f"Row= {row_sums}")
print(f"Col= {col_sums}")
```

INPUT

[[1,2],[3,4]]

OUTPUT

(no output)

Q6. Maximum Subarray Sum

Score: 6.67 TC: 2/3 passed

CODE

```
def max_subarray_sum(nums):
    curr = res = nums[0]
    for x in nums[1:]:
        curr = max(x, curr + x)
        res = max(res, curr)
    return res
print(max_subarray_sum(list(map(int, input().split()))))
```

INPUT

-1 -2 -3

OUTPUT

-1

Q7. Common Elements Among Three Lists

Score: 10.00 TC: 3/3 passed

CODE

```
try:
    inputs = input().split(';')
    list1 = list(map(int, inputs[0].split()))
    list2 = list(map(int, inputs[1].split()))
    list3 = list(map(int, inputs[2].split()))
    common = list(set(list1) & set(list2) & set(list3))
    print(sorted(common))
except EOFError:
    pass
```

INPUT

1 2 3;2 3 4;2 5

OUTPUT

[2]

Q8. Matrix Multiplication

```

# Convert parts to integers and list
a = list(map(int, list_part.split()))
k = int(k_part)

# 1. Sort the list
a.sort()
n = len(a)

# 2. Perform Rotation (Right Rotation)
if n > 0:
    k = k % n
    # Right rotation: take the last k elements and put them at the front
    rotated = a[-k:] + a[:-k] if k != 0 else a[:]
else:
    rotated = a

# 3. Identify Min and Max
val_min = min(a)
val_max = max(a)

# Printing with the exact required format: "Key= Value"
# Note: Python's list printing [1, 2, 3] includes spaces after commas by default
print(f"Sorted= {a}")
print(f"Rotated= {rotated}")
print(f"Min= {val_min}")
print(f"Max= {val_max}")

except Exception as e:
    pass # Silently handle parsing errors if input is malformed

```

INPUT

List=3 1 2,k=1

OUTPUT

```

Sorted= [1, 2, 3]
Rotated= [3, 1, 2]
Min= 1
Max= 3

```


Q10. Sort Matrix Search

Score: 0.00 TC: 0/3 passed

CODE

```
matrix = eval(input("Enter matrix: "))
find = int(input("Enter number to find: "))

elements = []
for row in matrix:
    elements.extend(row)

elements.sort()

rows = len(matrix)
cols = len(matrix[0])

sorted_matrix = []
for i in range(rows):
    sorted_matrix.append(elements[i * cols:(i + 1) * cols])

print("Sorted =", sorted_matrix)

if find in elements:
    index = elements.index(find)
    row = index // cols
    col = index % cols
    print("Position =", (row, col))
else:
    print("Not Found")
```

INPUT

```
[[9,3],[7,1]]
7
```

OUTPUT

Security Error: Disallowed code pattern detected.